Проєкт для дисципліни

Інформаційне забезпечення промислових роботів

Solving Battlesnake with Behavior Cloning, Maskable PPO, and Self-Play

An Implementation-Focused Reinforcement Learning Report

Abstract

We study two-player Standard Battlesnake on an 11×11 grid. A convolutional agent first learns from 150,000 simulated teacher state–action examples by behavior cloning and then improves through 10 million agent transitions using Maskable Proximal Policy Optimization (PPO). The training pool combines scripted opponents with frozen self-play snapshots. In a held-out, frozen-model evaluation of 4,000 duels against four specified scripted opponents, the agent wins 3,737 games (93.425%). This document details the state encoding, action masking, network, loss functions, training sequence, evaluation methods, and measured results, using editable LaTeX figures.

Scope. This report reconstructs the implemented Battlesnake system from the project report, including the architecture, training configuration, evaluation protocol, and recorded results. Illustrative diagrams are schematic, not captured simulator frames. Derived quantities are explicitly labeled as such.

October 2026

Contents

1 Problem Formulation **3**
- 1.1 Battlesnake Duel Environment . . . 3
- 1.2 Two-Player Stochastic Game . . . 3
- 1.3 Reward Function and Objective . . . 3

2 Environment and State Representation **4**
- 2.1 Gymnasium–Go Integration . . . 4
- 2.2 Observation Tensor . . . 4
- 2.3 Action Masking . . . 4

3 Neural Network Architecture **5**
- 3.1 Shared Convolutional Feature Extractor . . . 5
- 3.2 Actor and Critic Heads . . . 6
- 3.3 Receptive Field and Parameters . . . 6

4 Scripted Heuristic Opponents **7**
- 4.1 Rule-Based Decision Making . . . 7
- 4.2 Five Opponent Strategies . . . 7
- 4.3 Connection to Behavior Cloning and PPO . . . 8

5 Behavior Cloning Initialization **8**
- 5.1 Simulated Demonstrations . . . 8
- 5.2 Supervised Objective . . . 8
- 5.3 Why Imitation Pretraining? . . . 9

6 Maskable Proximal Policy Optimization **9**
- 6.1 Rollout Collection and Advantages . . . 9
- 6.2 Clipped Policy Objective . . . 10
- 6.3 Value Loss, Entropy, and Total Loss . . . 10
- 6.4 Training Schedule and Safeguards . . . 10

7 Self-Play and Full Training Pipeline **11**
- 7.1 Opponent Pool and Self-Play Introduction . . . 11
- 7.2 End-to-End Schedule . . . 11
- 7.3 Evaluation, Checkpoints, and Runtime Architecture . . . 12

8 Experimental Setup and Results **12**
- 8.1 Evaluation Protocol . . . 12
- 8.2 Performance Against Scripted Opponents . . . 13
- 8.3 Inference Latency . . . 13

Conclusion **14**

1 Problem Formulation

1.1 Battlesnake Duel Environment

We consider *Standard 1v1 Battlesnake* played on a board of height $H = 11$ and width $W = 11$. Each player controls a snake and chooses one of four cardinal actions each turn:

$$\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}, \qquad |\mathcal{A}| = 4. \tag{1}$$

The two moves are applied *simultaneously* by the official Go simulator. Eating food increases snake length and restores health to 100; otherwise health decreases by one per turn. A snake loses by moving outside the grid, colliding with a body, or exhausting its health. In a head-to-head collision the longer snake survives, while equal-length heads eliminate one another. Experimental games are capped at 1,000 turns.

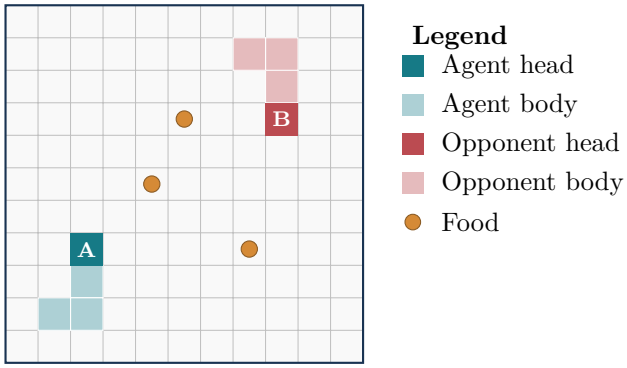


Figure 1: Schematic 11×11 duel board. The positions are illustrative and not part of the measured evaluation.

1.2 Two-Player Stochastic Game

Let $s_t \in \mathcal{S}$ be the full board state. The learning agent selects $a_t \in \mathcal{A}$ while the opponent selects $b_t \in \mathcal{A}$. The environment then evolves according to

$$s_{t+1} \sim P(\cdot \mid s_t, a_t, b_t). \tag{2}$$

Our agent does not directly process the full simulator state. It receives an encoded observation $\mathbf{o}_t = f(s_t)$ together with a binary legal-action filter $\mathbf{m}_t \in \{0, 1\}^4$. The policies can be expressed as

$$a_t \sim \pi_\theta(\cdot \mid \mathbf{o}_t, \mathbf{m}_t), \qquad b_t \sim \pi_{\text{opp}}(\cdot \mid s_t). \tag{3}$$

The opponent policy changes over training because opponents are sampled from scripted strategies and, later, saved versions of the learning agent.

1.3 Reward Function and Objective

The reported implementation uses the following sparse outcome reward, with a small food-collection incentive:

$$r_t = \begin{cases} +1, & \text{the agent wins at a terminal turn,} \\ -1, & \text{the agent loses at a terminal turn,} \\ 0, & \text{both snakes are eliminated,} \\ +0.005, & \text{food is eaten on a nonterminal turn,} \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

Terminal outcome rewards take precedence over the food bonus. For an episode ending after $T \leq 1000$ turns, the objective is to maximize expected discounted return:

$$J(\theta) = \mathbb{E}_{\pi_\theta, \pi_{\text{opp}}, P}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right], \qquad \gamma = 0.995. \tag{5}$$

The agent is initialized through imitation of a scripted teacher and subsequently updated from its own gameplay through PPO. These are different learning objectives: supervised prediction of teacher moves followed by reward maximization.

2 Environment and State Representation

2.1 Gymnasium–Go Integration

The official Battlesnake simulator implements the rules in Go, while Python provides a Gymnasium-compatible training interface. The `SnakeEnv` interface exposes three central methods: `reset()`, `step(action)`, and `action_masks()`. Communication passes through a JSON bridge. The environment returns observations, rewards, termination and truncation flags, and auxiliary information; the learning code updates the weights separately.

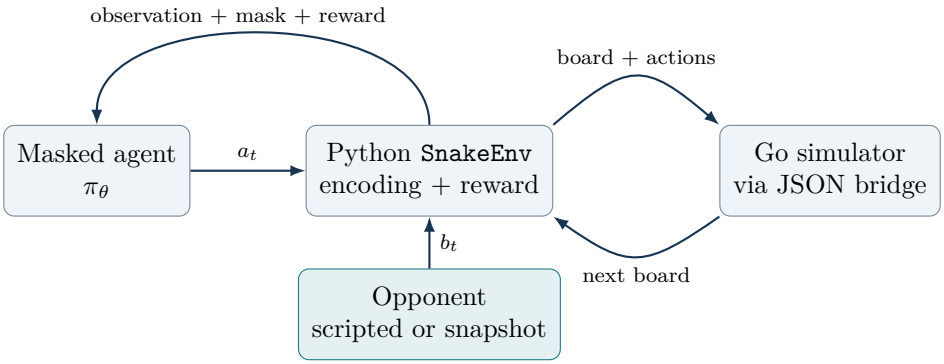


Figure 2: Training interface: the Python environment connects model decisions and opponents to the Go rules engine.

2.2 Observation Tensor

Every observation is stored as a 31-channel tensor over the 11×11 board:

$$\mathbf{o}_t \in \{0, \ldots, 255\}^{31\times 11\times 11}, \qquad \tilde{\mathbf{o}}_t = \frac{\mathbf{o}_t}{255} \in [0,1]^{31\times 11\times 11}. \tag{6}$$

The channels represent food, hazards, the two snakes' heads and bodies, health, length, turn number, and opponent-head threat zones. The original report does not assign an exact index or channel count to each semantic group; therefore Figure 3 illustrates the *conceptual grouping*, not a claimed per-channel map. The dimensions correspond to $31 \times 121 = 3{,}751$ spatial input values.

2.3 Action Masking

For an action $a \in \mathcal{A}$, let $m_{t,a} = 1$ if the encoder does not identify it as a guaranteed immediate collision with a wall or body. Otherwise $m_{t,a} = 0$. Given actor logits $z_\theta(\mathbf{o}_t) \in \mathbb{R}^4$, the masked categorical policy

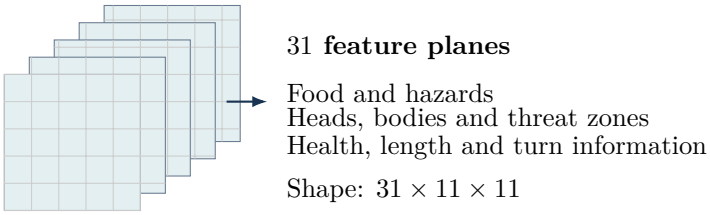


Figure 3: Schematic stack of input feature planes. Categories are known; exact channel-to-category indices are not specified by the source report.

is

$$\pi_\theta(a \mid \mathbf{o}_t, \mathbf{m}_t) = \frac{m_{t,a} \exp z_{\theta,a}(\mathbf{o}_t)}{\sum_{j \in \mathcal{A}} m_{t,j} \exp z_{\theta,j}(\mathbf{o}_t)}, \quad \text{when } \sum_j m_{t,j} > 0. \tag{7}$$

Thus an excluded action has exactly zero probability. If all four actions are excluded, the implementation re-enables all four to keep the distribution defined; it does *not* claim that any move is safe. The mask detects immediate known collisions, not every future trap or head-to-head danger.

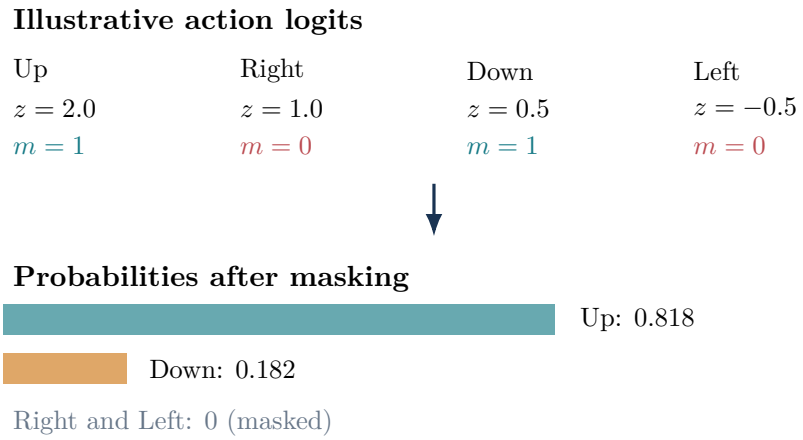


Figure 4: Numerical masking example, using $z = (2, 1, 0.5, -0.5)$ and $m = (1, 0, 1, 0)$. The values are illustrative, not measured policy outputs.

3 Neural Network Architecture

3.1 Shared Convolutional Feature Extractor

The agent uses a compact CNN with two 3×3 convolutions and a fully connected projection. For one observation, the forward computation has the following shapes:

$$\tilde{\mathbf{o}} \in \mathbb{R}^{31 \times 11 \times 11}, \tag{8}$$

$$h_1 = \mathrm{ReLU}(\mathrm{Conv}^{3\times3}_{31\to32}(\tilde{\mathbf{o}})) \in \mathbb{R}^{32 \times 11 \times 11}, \tag{9}$$

$$h_2 = \mathrm{ReLU}(\mathrm{Conv}^{3\times3}_{32\to32}(h_1)) \in \mathbb{R}^{32 \times 11 \times 11}, \tag{10}$$

$$h = \mathrm{ReLU}(W_f \,\mathrm{Flatten}(h_2) + b_f) \in \mathbb{R}^{128}. \tag{11}$$

Both convolutions use stride 1 and padding 1, so they preserve the spatial dimensions. Flattening produces $32 \cdot 11 \cdot 11 = 3{,}872$ features before the linear projection to 128.

3.2 Actor and Critic Heads

The same hidden representation feeds two independent linear outputs:

$$z_\theta(\mathbf{o}) = W_\pi h + b_\pi \in \mathbb{R}^4, \qquad V_\theta(\mathbf{o}) = w_V^\top h + b_V \in \mathbb{R}. \tag{12}$$

The actor provides action logits which become the masked distribution in Eq. (7). The critic estimates expected discounted return and is used to compute advantages during PPO. Action masking constrains the actor's distribution but does not remove the critic's value estimate.

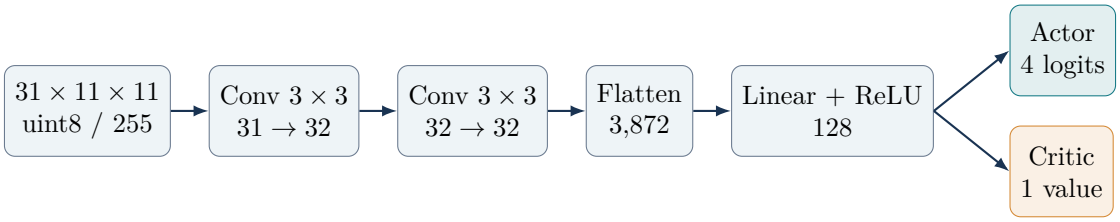


Figure 5: CNN shared backbone and two heads. The policy head has four action logits; the critic head estimates a single state value.

3.3 Receptive Field and Parameters

With stride 1 and 3×3 kernels, the receptive field of one feature grows from 1×1 input position to 3×3 after one convolution and 5×5 after two. However, the subsequent flatten-and-linear layer combines all spatial positions, enabling the final policy to depend on the whole board.

Layer	Output shape	Trainable parameters
Conv $31 \to 32$, 3×3	$32 \times 11 \times 11$	8,960
Conv $32 \to 32$, 3×3	$32 \times 11 \times 11$	9,248
Linear $3{,}872 \to 128$	128	495,744
Actor linear $128 \to 4$	4	516
Critic linear $128 \to 1$	1	129
Total		514,597

Table 1: Parameter count derived from the layer shapes, including biases; agrees with the project report.

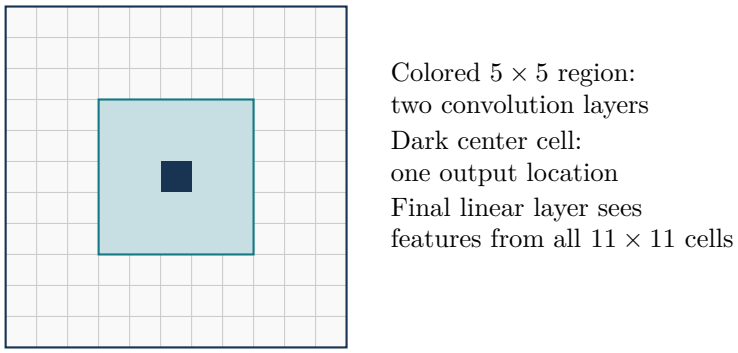


Figure 6: Receptive field of a central second-layer feature. The 5×5 locality applies to a single spatial feature, not to the final policy prediction.

4 Scripted Heuristic Opponents

4.1 Rule-Based Decision Making

Before reinforcement learning, the simulator provides several deterministic or rule-driven opponents. Unlike the learned CNN policy, a scripted opponent follows hand-designed navigation and survival rules rather than optimizing neural-network parameters. These policies play two roles: the `teacher` generates demonstrations for behavior cloning, and a collection of scripted policies supplies diverse adversaries during PPO training.

For a board state s_t, a generic heuristic can be written as

$$a_t^{(h)} = h(s_t), \qquad a_t^{(h)} \in \mathcal{A}, \tag{13}$$

where h is a fixed decision procedure. A rule-based opponent may compare candidate next cells using immediate collision risk, distances to food, space available for escape, or proximity to the opposing snake. These are complementary signals: the nearest food is not always the safest destination, and a move that is safe this turn may lead into a dead end.

4.2 Five Opponent Strategies

The training setup contains five named scripted opponents. Their *high-level roles* are summarized in Table 2. These descriptions indicate the strategy each opponent is intended to emphasize; they are not a claim about exact, undocumented scoring coefficients or tie-breaking rules.

Opponent	Main decision principle
`teacher`	Breadth-first search (BFS) for navigation, combined with hand-designed survival and movement heuristics; used as the imitation-learning teacher.
`heuristic`	General-purpose scripted player balancing immediate danger avoidance with searching for food.
`teacher_cautious`	A more conservative teacher variant that puts greater emphasis on avoiding hazardous positions.
`flood_hungry`	A space-oriented policy that favors moves leaving a large region of reachable free cells (flood-fill intuition).
`greedy_food`	A food-seeking policy that prioritizes moving toward food, rather than optimizing long-term board control.

Table 2: Qualitative functions of the five scripted opponents. Precise priorities and tie-breakers depend on the implementation.

Two common building blocks help explain the difference between the policies. A shortest-path food-seeking heuristic can compare the graph distance from the next head position $h_{t+1}(a)$ to food cells $f \in \mathcal{F}_t$:

$$D_{\text{food}}(a) = \min_{f \in \mathcal{F}_t} d_{\text{BFS}}\big(h_{t+1}(a), f\big). \tag{14}$$

A space-seeking heuristic can instead compare the number of cells reachable without crossing currently blocked cells:

$$R_{\text{space}}(a) = \big|\text{Reachable}\big(h_{t+1}(a); s_t\big)\big|. \tag{15}$$

These formulas illustrate shortest-path and flood-fill *principles*; they do not specify an exact combined scoring function used by any single opponent. Food locations, occupancy and available routes can change after the simultaneous moves of both snakes, so neither signal alone guarantees survival.

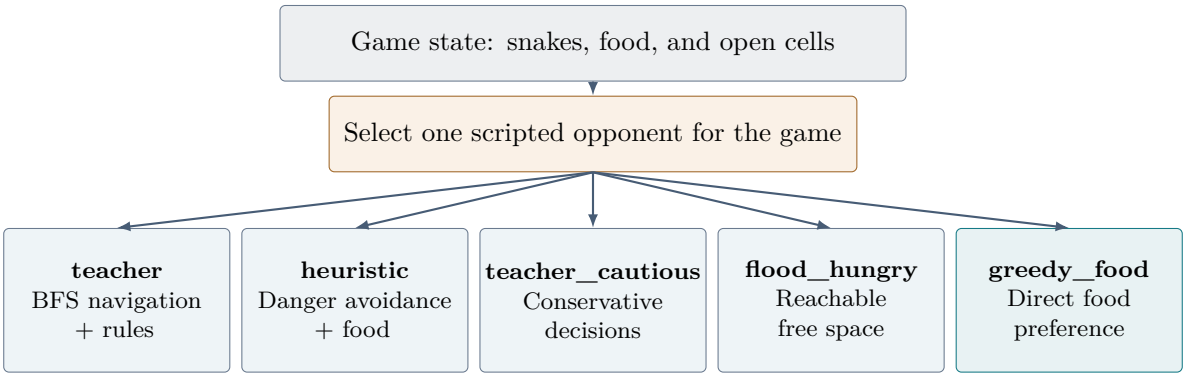


Figure 7: Conceptual opponent pool. The labels summarize different strategic emphases rather than individual code branches or exact score functions.

4.3 Connection to Behavior Cloning and PPO

The demonstrations are generated by the `teacher`, so behavior cloning learns to approximate that scripted decision rule on observed states. The imitation model is then optimized against the broader scripted opponent pool with Maskable PPO. Different opponents expose different failure modes: aggressive food collection can be punished by traps, while conservative space management can concede food or board control. Later, frozen copies of the learned policy join the opponent pool as self-play opponents; the scripted heuristics remain useful baseline strategies for evaluation.

5 Behavior Cloning Initialization

5.1 Simulated Demonstrations

No external demonstration dataset is used. The project generates a dataset by running a scripted teacher in the simulator. Each record contains the observed state, a teacher-selected action, and an action mask:

$$\mathcal{D}_{\mathrm{BC}} = \{(\mathbf{o}_i, a_i^*, \mathbf{m}_i)\}_{i=1}^{N}, \qquad N = 150{,}000. \tag{16}$$

About 90% of *games* are assigned to training and 10% to validation. Splitting by games rather than by individual frames reduces leakage between near-identical consecutive states. The number 150,000 counts collected state–action examples, not necessarily independent episodes.

5.2 Supervised Objective

The supervised policy learns to predict teacher actions. The standard categorical cross-entropy objective is

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\frac{1}{N_{\mathrm{tr}}} \sum_{i \in \mathcal{I}_{\mathrm{tr}}} \log \pi_\theta(a_i^* \mid \mathbf{o}_i, \mathbf{m}_i). \tag{17}$$

The stored masks make masked prediction possible. The original prose report identifies cross-entropy as the training loss but does not explicitly document whether the masks are applied inside the BC loss; Eq. (17) shows the mask-conditioned form consistent with the eventual policy interface. The report does not establish that the mask was included in the supervised loss.

Training uses Adam with learning rate 3×10^{-4} for up to 15 epochs. The checkpoint with the lowest validation loss is retained. The imitation-only policy is also evaluated as a candidate; its weights

initialize the network used for reinforcement learning.

Behavior Cloning setting	Value
Demonstration source	Scripted teacher in simulated games
Dataset size	150,000 state–action–mask examples
Training / validation split	Approximately 90% / 10% (by game)
Input observation	$31 \times 11 \times 11$, stored as `uint8`
Prediction target	One of four teacher-selected moves
Training objective	Categorical cross-entropy
Optimizer	Adam
Learning rate	3×10^{-4}
Maximum training epochs	15
Checkpoint criterion	Lowest validation loss
Downstream use	Initialize Maskable PPO

Table 3: Reported behavior cloning configuration. Values are taken from the project description; an unreported minibatch size or other optimizer settings are not assumed.

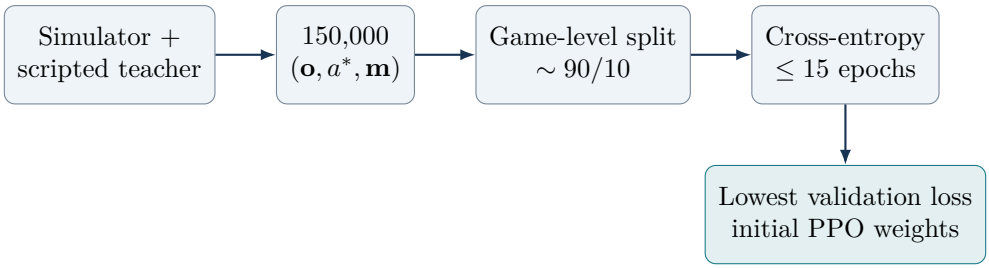


Figure 8: Behavior cloning pipeline. The imitation checkpoint supplies the initialization for subsequent PPO training.

5.3 Why Imitation Pretraining?

A policy trained from scratch may spend considerable time discovering basic movement and food collection. Behavior cloning supplies a useful initial decision rule, while PPO can improve beyond the teacher by optimizing rewards from full games. The approaches are complementary: BC minimizes disagreement with recorded actions; PPO maximizes expected return against the encountered opponent distribution.

6 Maskable Proximal Policy Optimization

6.1 Rollout Collection and Advantages

The agent collects fresh on-policy experience under its current *masked* policy. For each transition it stores an observation, action, valid-action mask, reward, estimated value, and the log-probability under the behavior policy. With $n_{\text{env}} = 16$ parallel games and at most $n_{\text{steps}} = 256$ steps per environment, a full rollout contains

$$N_{\text{rollout}} = 16 \cdot 256 = 4{,}096 \text{ agent transitions.} \tag{18}$$

Let $V_\theta(\mathbf{o}_t)$ denote the critic estimate. Generalized Advantage Estimation (GAE) computes temporal-difference residuals and advantage estimates:

$$\delta_t = r_t + \gamma(1 - d_t)V_\theta(\mathbf{o}_{t+1}) - V_\theta(\mathbf{o}_t), \tag{19}$$

$$\widehat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \qquad \gamma = 0.995, \quad \lambda = 0.95. \tag{20}$$

Here d_t indicates a genuine terminal transition; truncation and bootstrapping require the implementation's corresponding handling. Conceptually, $\widehat{A}_t$ estimates whether the taken action performed better or worse than the critic expected.

6.2 Clipped Policy Objective

For a rollout action a_t, define the importance ratio between the updated and collecting policies as

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid \mathbf{o}_t, \mathbf{m}_t)}{\pi_{\theta_{\text{old}}}(a_t \mid \mathbf{o}_t, \mathbf{m}_t)}. \tag{21}$$

The mask is stored with the transition and reused when evaluating its action, keeping numerator and denominator on the same allowed action set. PPO minimizes the negative clipped surrogate:

$$\mathcal{L}_{\text{policy}}(\theta) = -\mathbb{E}_t\left[\min\left(\rho_t(\theta)\widehat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t\right)\right], \quad \epsilon = 0.15. \tag{22}$$

The clipping interval is $[0.85, 1.15]$. This clips the objective's probability ratio; it is *not* a strict guarantee that every action probability can move by at most 15% during optimization.

6.3 Value Loss, Entropy, and Total Loss

The critic learns the target return $\widehat{R}_t = \widehat{A}_t + V_{\theta_{\text{old}}}(\mathbf{o}_t)$. A generic clipped-value error, with value clipping threshold $\epsilon_V = 0.2$, is

$$V_t^{\text{clip}}(\theta) = V_{\text{old},t} + \text{clip}\big(V_\theta(\mathbf{o}_t) - V_{\text{old},t}, -\epsilon_V, \epsilon_V\big), \tag{23}$$

$$\mathcal{L}_V(\theta) = \frac{1}{2}\mathbb{E}_t\Big[\max\Big\{(V_\theta(\mathbf{o}_t) - \widehat{R}_t)^2, (V_t^{\text{clip}}(\theta) - \widehat{R}_t)^2\Big\}\Big]. \tag{24}$$

The masked action entropy is $H_t = -\sum_{a\in\mathcal{A}} \pi_\theta(a \mid \mathbf{o}_t, \mathbf{m}_t) \log \pi_\theta(a \mid \mathbf{o}_t, \mathbf{m}_t)$. The complete optimization has policy, value, and entropy terms:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{policy}}(\theta) + c_V\mathcal{L}_V(\theta) - c_H\mathbb{E}_t[H_t], \qquad c_V = 0.5, \quad c_H = 0.01. \tag{25}$$

Some libraries absorb a factor of $1/2$ into the definition of the value loss. Equation (25) makes that convention explicit; the exact numerical reduction follows the PPO implementation. The entropy term rewards exploration among *allowed* actions.

6.4 Training Schedule and Safeguards

The reported PPO optimizer is Adam with learning rate 10^{-4}. Rollouts are replayed for up to four training epochs using minibatches of 256. A full rollout has $4096/256 = 16$ minibatches, corresponding to *up to* 64 optimizer minibatch updates if every PPO epoch completes. Early stopping is triggered if approximate KL divergence exceeds $1.5 \cdot 0.02 = 0.03$, and additional numerical checks may also stop updates. Gradients are clipped to a global norm of 0.5. New rollouts are collected after the current batch is updated, as PPO is an on-policy method.

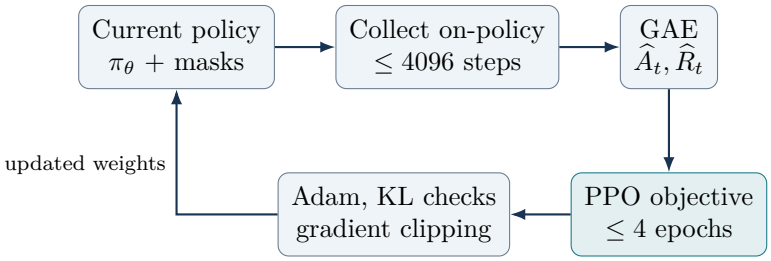


Figure 9: One repeated Maskable PPO cycle. Policy weights are updated after rollout collection, not after every simulator move.

Setting	Value
Optimizer / PPO learning rate	Adam / 10^{-4}
Discount γ / GAE λ	0.995 / 0.95
Policy clip / value clip	0.15 / 0.2
Entropy / value coefficient	0.01 / 0.5
Target KL / stop threshold	0.02 / 0.03
Maximum gradient norm	0.5
Environments / worker processes	16 / 2
Rollout length per environment	Up to 256
Full rollout / minibatch	4,096 / 256
Training epochs per rollout	Up to 4
Training seed / total transitions	1 / 10,000,000

Table 4: Reported Maskable PPO optimization and experience-collection settings.

7 Self-Play and Full Training Pipeline

7.1 Opponent Pool and Self-Play Introduction

PPO training uses randomized starting positions and an opponent pool containing five scripted strategies: `teacher`, `heuristic`, `teacher_cautious`, `flood_hungry`, and `greedy_food`. After 200,000 agent transitions, saved snapshots of our own model become eligible opponents in up to 30% of games. Thus self-play is introduced *within* the overall PPO training run, not necessarily as a completely separate training run.

The exact individual sampling weights of the five scripted opponents and the snapshot-selection rule are not specified in the source report. We therefore represent the opponent mixture abstractly by

$$\pi_{\text{opp},t} \sim \begin{cases} \mathcal{P}_{\text{scripted}}, & t < 200{,}000, \\ (1-p_t)\mathcal{P}_{\text{scripted}} + p_t \mathcal{P}_{\text{snapshots}}, & t \geq 200{,}000, \end{cases} \quad 0 \leq p_t \leq 0.30. \tag{26}$$

Here the mixture notation describes how an opponent is selected for a game, not a parameter-wise average of policies. The threshold corresponds to 2% of the stated 10-million-transition budget. Opponents frozen from earlier checkpoints provide variety and may reduce exploitation of a single predictable adversary.

7.2 End-to-End Schedule

The complete learning trajectory combines supervised pretraining with long-horizon interaction learning:

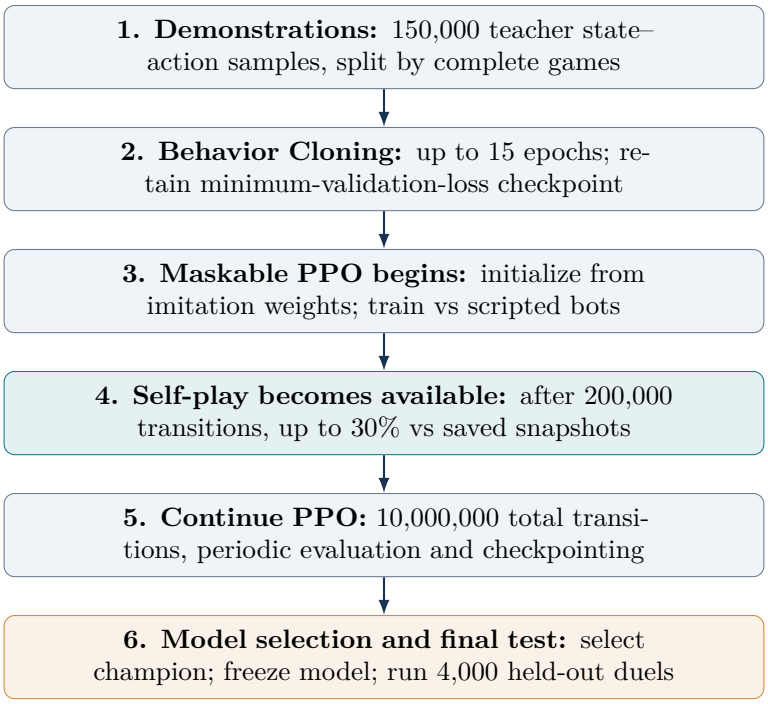


Figure 10: Reported full training sequence. The 10 million transitions cover PPO as a whole, including its self-play-enabled phase.

7.3 Evaluation, Checkpoints, and Runtime Architecture

The run checks performance every 100,000 transitions using 400 validation games per check. The report describes separate 2,000-game control series for champion selection. The selected policy is then frozen before a distinct final 4,000-game evaluation, so the final games are not used to change its weights. Starting positions are paired on the same board with sides swapped for fairness.

The runtime connects the Go game simulator, a Gymnasium-compatible environment, batched game collection, the policy and value networks, and an evaluation system. Periodic checkpoints preserve the state needed to resume training. After champion selection, the policy is exported to TorchScript for deployment through an HTTP endpoint and a browser-based arena.

8 Experimental Setup and Results

8.1 Evaluation Protocol

The target was to achieve at least 70% wins against the selected heuristic opponents. After model selection, the frozen champion played 4,000 final duels. The four evaluated opponents were `teacher`, `heuristic`, `teacher_cautious`, and `flood_hungry`, each appearing in an equal share of games. This is *not* an evaluation over all training-time opponents: `greedy_food` appears in the training pool but not in the four-opponent final results reported here.

Let w_i be the indicator that duel i ends in an agent win. The estimated win rate is

$$\widehat{p}_{\text{win}} = \frac{1}{N}\sum_{i=1}^{N} w_i = \frac{3{,}737}{4{,}000} = 0.93425 \quad (93.425\%). \tag{27}$$

The source report gives the associated 95% confidence interval as $[92.675\%, 94.175\%]$. The confidence-interval method is not documented in the prose, so this is reported rather than re-derived.

8.2 Performance Against Scripted Opponents

Opponent	Duels	Agent wins	Win rate
`teacher`	1,000	951	95.1%
`heuristic`	1,000	928	92.8%
`teacher_cautious`	1,000	926	92.6%
`flood_hungry`	1,000	932	93.2%
Total	4,000	3,737	93.425%

Table 5: Final frozen-model evaluation. Per-opponent game and win counts follow from the reported equal shares and win rates.

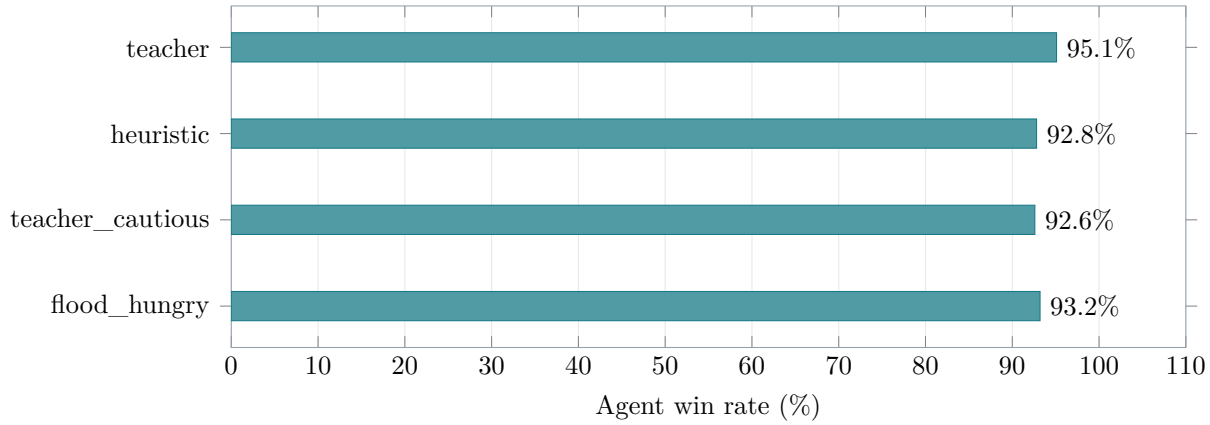


Figure 11: Final win rates by opponent (frozen model, 1,000 reported games per opponent). Labels reproduce the results table, not training-time learning curves.

The overall result exceeds the 70% target by 23.425 percentage points. This validates performance against the four chosen scripted opponents under the tested rules; it does not imply an equally high win rate against human players, stronger learned agents, or different board configurations.

8.3 Inference Latency

The original final evaluation reported no move exceeding 500 ms and a measured $p95$ decision latency of roughly 0.875 ms *in that evaluation environment*. Separately, the project report describes local CPU timing measurements collected on 22 October 2026 using an Intel Core i5-10210U, one CPU thread, PyTorch 2.8.0, batch size 1. Both local timings use 5,120 calls over 512 states collected from 12 games, after 200 warm-up calls.

Because the hardware and timing boundaries differ, the 0.875 ms evaluation $p95$ should not be mixed with the local 2.238 ms complete-decision $p95$ as though they were measured under the same conditions.

Local CPU timed operation	**Median**	$p95$
Neural forward pass only	0.708 ms	1.482 ms
Complete policy decision	1.082 ms	2.238 ms

Table 6: Local CPU latency. Complete decision includes encoding, neural inference, masking, and argmax. Both exclude model loading, Go simulation, HTTP, and rendering.

Conclusion

An 11×11 Battlesnake agent was first initialized with behavior cloning and then trained with Maskable PPO for 10 million agent transitions. The model uses a 514,597-parameter shared CNN, a categorical policy head constrained by action masks, and a scalar critic. Training first emphasizes scripted opponents and later introduces saved self-play snapshots in up to 30% of games. A frozen final benchmark recorded 3,737 wins across 4,000 duels (93.425%), comfortably above the 70% target within the tested setting. Follow-up research should examine performance across seeds, stronger opponents, and component ablations.

Provenance. Technical settings, experiment counts, and results were reconstructed from the provided six-page project report, *Solving Battlesnake with Reinforcement Learning*. Explanatory mathematics and schematic illustrations were added for pedagogy; unreported implementation choices are not invented.